

Sistemas Distribuidos

Guía para crear un microservicio que permite realizar callback mediante webSockets

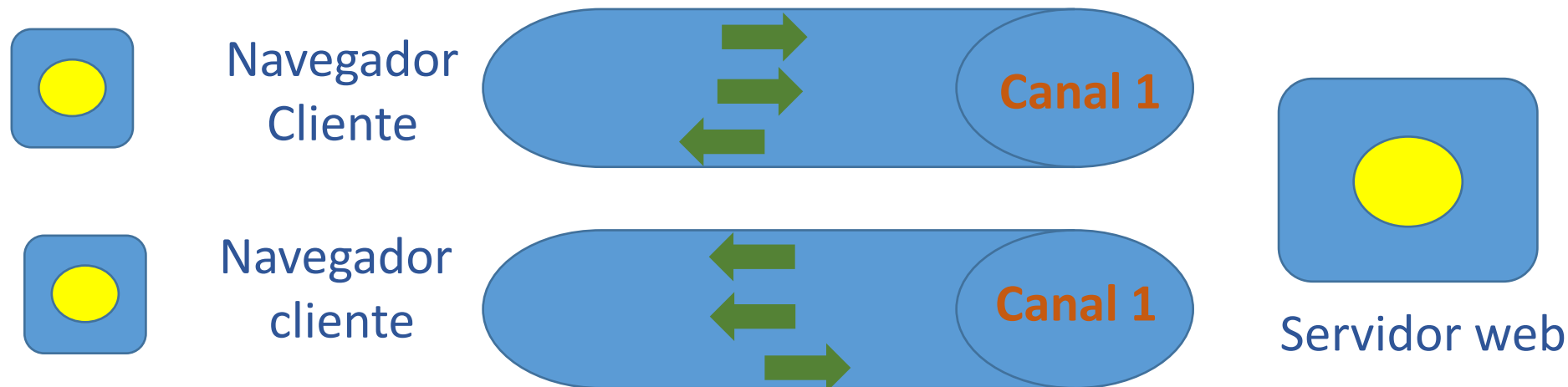
PROGRAMA DE INGENIERÍA DE SISTEMAS

Ing. Daniel Eduardo Paz Perafán (danielp@Unicauca.edu.co)

WebSockets

WebSockets es un protocolo que hace posible abrir una sesión de comunicación interactiva entre el navegador y un servidor. Permite enviar mensajes a un servidor y recibir respuestas controladas por eventos sin tener que consultar al servidor. **Estándar oficial RFC 6455.**

A diferencia del protocolo HTTP tradicional, donde cada solicitud requiere una nueva conexión, WebSocket mantiene una única conexión abierta, lo que facilita el intercambio de datos en tiempo real con menor latencia y sobrecarga.



Protocolos utilizados

HTTP/1.1 (Solo para el inicio de la conexión)

Función: El proceso de establecimiento de una conexión WebSocket comienza con un "handshake" (apretón de manos) que utiliza una solicitud HTTP/1.1 especial.

Esta solicitud incluye las cabeceras Upgrade: websocket y Connection: Upgrade, indicando al servidor la intención de cambiar al protocolo WebSocket.

Importancia: Este mecanismo permite que WebSocket sea compatible con la infraestructura web existente, como proxies y firewalls, al utilizar los mismos puertos estándar (80 y 443).

Protocolos utilizados

TCP (Capa de transporte principal)

Función: Una vez completado el handshake, la comunicación entre cliente y servidor se realiza sobre una conexión TCP persistente, debido a que proporciona una transmisión de datos confiable y ordenada, asegurando que los mensajes lleguen completos y en orden correcto.

Importancia: La persistencia de la conexión TCP permite una comunicación continua sin la necesidad de establecer nuevas conexiones para cada mensaje, lo que reduce la latencia y mejora la eficiencia

Protocolos utilizados

TLS/SSL (Para conexiones seguras)

Función: Cuando se requiere una conexión segura, WebSocket utiliza TLS/SSL para cifrar los datos transmitidos.

Importancia: El uso de TLS/SSL garantiza la confidencialidad e integridad de los datos, protegiendo la comunicación contra interceptaciones y ataques.

Protocolos utilizados

WebSocket (Protocolo de aplicación)

Función: Después de establecer la conexión, el protocolo WebSocket define el formato de los mensajes intercambiados entre el cliente y el servidor. Los datos se transmiten en "frames" (tramas) que pueden contener texto, datos binarios o señales de control.

Importancia: Este protocolo permite una comunicación full-duplex, donde ambas partes pueden enviar y recibir datos simultáneamente, facilitando aplicaciones en tiempo real como chats, juegos en línea y actualizaciones en vivo.

Protocolos utilizados

Protocolo STOMP (Streaming Text Oriented Messaging Protocol)

Función: Es el protocolo usado sobre WebSockets para enviar y recibir mensajes en aplicaciones como chats, notificaciones en tiempo real.

Importancia: Permite realizar las siguientes actividades:

- Conectarte (connect).
- Suscribirse a canales (subscribe).
- Enviar mensajes (send).
- Desconectarte (disconnect).

Introducción a la practica

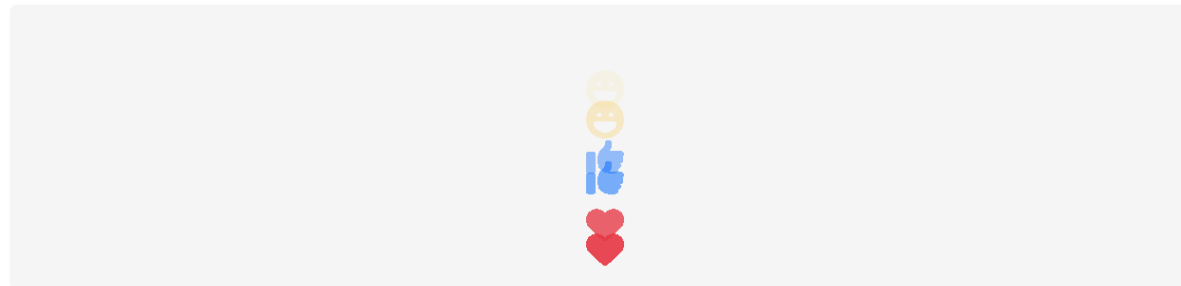
UNIVERSIDAD DEL CAUCA – FIET
DEPARTAMENTO DE SISTEMAS

En esta practica se debe crear un microservicio que permita conectar a varios clientes mediante webSockets. Luego de la conexión, los clientes pueden enviar reacciones asociadas a las canciones que están reproduciendo. El microservicio reenviará la reacción a los clientes conectados mediante un callBack y utilizando la conexión previamente creada.

Cuando el cliente da click sobre uno de los iconos, se envía la reacción al microservicio.



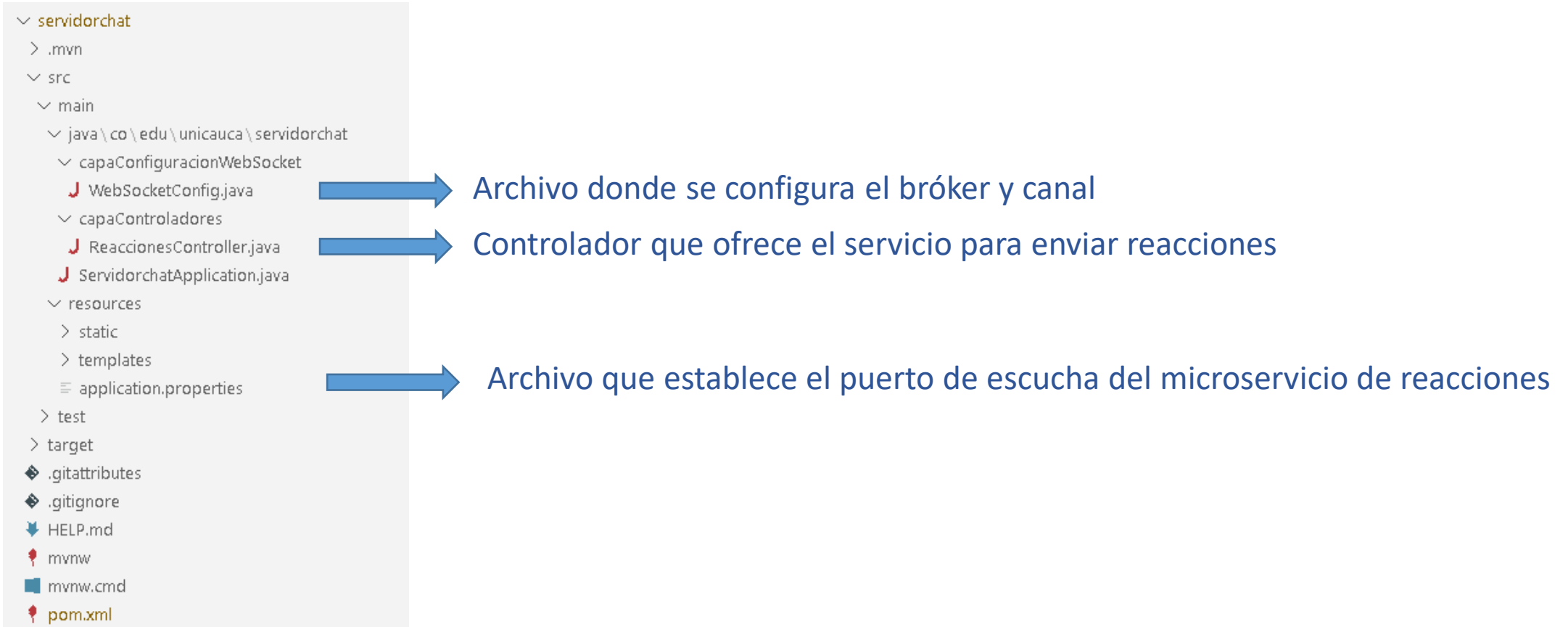
Reacciones recibidas:

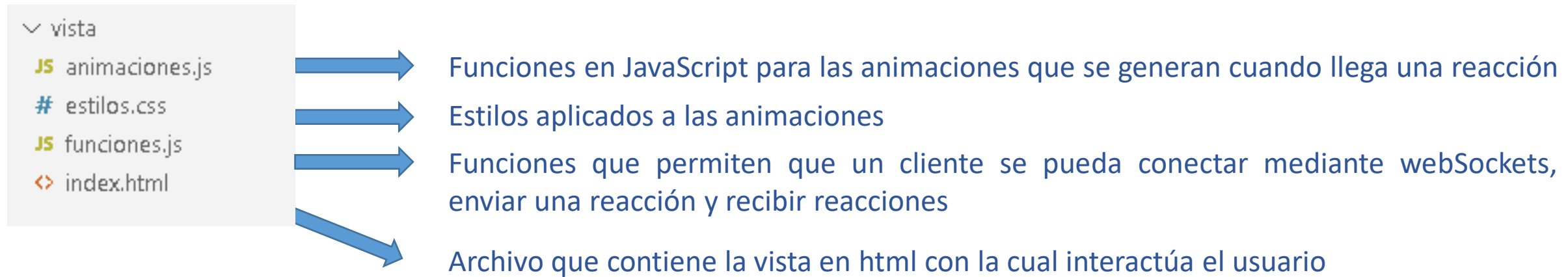


Cuando el microservicio recibe la reacción, la reenvía a los demás clientes conectados.

La solución de este ejercicio debe ser comprimida. El nombre del archivo comprimido debe seguir el siguiente formato `lsd_rpc_p7_apellido1N_apellido2N.rar`. Donde apellido1 corresponde al primer apellido de uno de los integrantes, más la inicial del Primer Nombre y apellido2 corresponde al primer apellido del segundo integrante del grupo más la inicial del Primer Nombre. **Dentro de la carpeta se debe establecer en un archivo los roles de cada uno de los integrantes.**

Paquetes del microservicio de reacciones





Función para establecer la conexión con el microservicio de reacciones

UNIVERSIDAD DEL CAUCA – FIET
DEPARTAMENTO DE SISTEMAS

Variable que contiene la conexión mediante web Sockets con el servidor

```
let clienteChat = null;
```

Función que permite habilitar o des habilitar los botones de conexión y desconexión

```
function setConectado(conectado) {  
  document.getElementById('btnConectar').disabled = conectado;  
  document.getElementById('btnDesconectar').disabled = !conectado;  
}
```

Función que permite que el cliente se conecte mediante webSockets con el servidor que escucha por el puerto 5000

```
function conectar() {  
  const socket = new SockJS('http://localhost:5000/ws');  
  clienteChat = Stomp.over(socket);  
  
  clienteChat.connect({}, onConnected):  
}
```

Función que se invoca automáticamente cuando el cliente se conecta

Función para conectar el cliente con el microservicio de reacciones

La función `onConnected(frame)` se ejecuta cuando el cliente STOMP logra conectarse correctamente al servidor WebSocket.

```
function onConnected(frame) {  
  console.log("Conectado: " + frame);  
  clienteChat.subscribe('/brokerDeReacciones/reaccionesPorCancion', recibirReaccion);  
  setConectado(true);  
}
```

El cliente se suscribe a **/brokerDeReacciones/reaccionesPorCancion**

Esto significa que el cliente escuchará todos los mensajes que el servidor envíe a ese canal.

`recibirReaccion` es la función callback que se ejecuta cada vez que llegue una reacción.

Función para conectar el cliente con el microservicio de reacciones

UNIVERSIDAD DEL CAUCA – FIET
DEPARTAMENTO DE SISTEMAS

¿Por qué se llama **/brokerDeReacciones/reaccionesPorCancion?**

Porque se decidió agruparlo así:

- **/brokerDeReacciones:** nombre del canal principal
- **/reaccionesPorCancion:** subcanal específico para una canción

Esto permite tener varios topics, ejemplo:

- **/brokerDeReacciones/reaccionesPorCancion**
- **/brokerDeReacciones/usuariosConectados/**
- **brokerDeReacciones/reaccionesGlobales**

Función callback que recibe la reacción

```
function recibirReaccion(reaccion) {  
  console.log("Reacción reenviada:", reaccion.body);  
  const contenedor = document.getElementById("mensajes");  
  
  // Crear burbuja  
  const burbuja = document.createElement("div");  
  burbuja.classList.add("reaccion-burbuja");  
  
  let icono = "";  
  switch (reaccion.body) {  
    case "like":  
      icono = '<i class="fa-solid fa-thumbs-up icon-like"></i>';  
      break;  
    case "heart":  
      icono = '<i class="fa-solid fa-heart icon-heart"></i>';  
      break;  
    case "sad":  
      icono = '<i class="fa-solid fa-face-sad-tear icon-sad"></i>';  
      break;  
    case "fun":  
      icono = '<i class="fa-solid fa-face-laugh icon-fun"></i>';  
      break;  
  }  
  
  burbuja.innerHTML = icono;  
  contenedor.appendChild(burbuja);  
  // Eliminar burbuja después de la animación  
  setTimeout(() => {  
    burbuja.remove();  
  }, 3000); // coincide con la duración de la animación  
}
```

La función recibirReaccion es la encargada de manejar lo que ocurre cuando el cliente recibe una reacción enviada por el servidor a través del WebSocket/STOMP.

Se obtiene el contenedor donde aparecerá la animación

Se crea un div y se le asigna la clase que tiene la animación CSS (reaccion-burbuja)

Selecciona el icono según la reacción recibida

Se agrega la burbuja al contenedor

Se elimina la burbuja 3 segundos después

Selector que genera la animación cuando se recibe una reacción

Esa regla CSS define cómo aparece y cómo se mueve la burbuja de reacción, mezclando dos animaciones encadenadas:

```
.reaccion-burbuja {  
  position: absolute;  
  bottom: 0;  
  left: 50%;  
  font-size: 40px;  
  pointer-events: none;  
  opacity: 0;  
  transform: translate(-50%, 0) scale(0);  
  
  animation: pop 0.25s ease-out forwards,  
            flotar 3s ease-out forwards 0.25s;  
}
```

En conjunto, la animación hace:

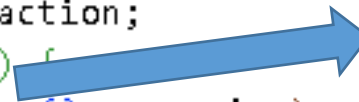
1. Aparecer / explotar la reacción como un pop suave
2. Luego la hace flotar hacia arriba siguiendo una curva
3. Luego se desvanece la reacción mientras sube
4. Finalmente la reacción queda en un estado final (opacidad 0)

Función para enviar la reacción al servidor

La función ejecuta la animación visual de la reacción (en el cliente local). Envía esa reacción al servidor por WebSocket/STOMP para que otros clientes la vean

```
function enviarReaccionServidor(event) {  
  const icon = event.target;  
  handleReactionClick(icon);  
  
  const reaccion = event.target.dataset.reaccion;  
  if (clienteChat && clienteChat.connected) {  
    clienteChat.send("/app/enviarReaccion", {}, reaccion);  
  } else {  
    alert("No estás conectado.");  
  }  
}
```

Esto envía la reacción al controlador, y luego el servidor la retransmite a todos los clientes suscritos.



Función para desconectar el cliente del servidor

La función desconectar() se encarga de cerrar correctamente la conexión WebSocket/STOMP entre el cliente y el servidor, y de actualizar el estado interno de la aplicación para indicar que el cliente ya no está conectado.

```
function desconectar() {  
  if (clienteChat !== null) {  
    clienteChat.disconnect(() => {  
      setConectado(false);  
    });  
    clienteChat = null;  
  }  
}
```

.disconnect() cierra la conexión WebSocket

El callback que se pasa (() => { setConectado(false); }) se ejecuta cuando la desconexión termina.

setConectado(false) actualiza la interface grafica.

```
<> index.html x
vista > <> index.html > html > body > div#reacciones > i.fa-solid.fa-thumbs-up.reaction.icon-like
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4    <meta charset="UTF-8">
5    <title>Spotify</title>
6    <script src="https://cdn.jsdelivr.net/npm/sockjs-client@1/dist/sockjs.min.js"></script>
7    <script src="https://cdn.jsdelivr.net/npm/stompjs@2.3.3/lib/stomp.min.js"></script>
8    <script src="funciones.js" defer></script>
9    <script src="animaciones.js" defer></script>
10   <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome@6.4.0/css/all.min.css">
11   <link rel="stylesheet" href="estilos.css">
12 </head>
13 <body>
14   <h2>Reproducción de canciones</h2>
15
16   <button id="btnConectar" onclick="conectar()">Conectar</button>
17   <button id="btnDesconectar" onclick="desconectar()" disabled>Desconectar</button>
18   <br><br>
19
20   <div id="reacciones">
21     <i class="fa-solid fa-thumbs-up reaction icon-like" data-reaction="like" onclick="enviarReaccionServidor(event)"></i>
22     <i class="fa-solid fa-heart reaction icon-heart" data-reaction="heart" onclick="enviarReaccionServidor(event)"></i>
23     <i class="fa-solid fa-face-sad-tear reaction icon-sad" data-reaction="sad" onclick="enviarReaccionServidor(event)"></i>
24     <i class="fa-solid fa-face-laugh reaction icon-fun" data-reaction="fun" onclick="enviarReaccionServidor(event)"></i>
25   </div>
26
27   <div id="chat">
28     <h3>Reacciones recibidas:</h3>
29     <div id="mensajes">
30       <!-- Las reacciones se mostrarán aquí -->
31     </div>
32   </div>
33 </body>
```

Librerías que permiten usar
el protocolo webSocket
mediante el protocolo stomp

Librerías de iconos

Boton que permite conectar al
cliente mediante webSocket

Al dar click en el icono se
debe ejecutar la función
enviarReaccionServidor

Contenedor donde se
reciben los mensajes

Controlador que recibe las reacciones y las reenvía a los suscritos a
/brokerDeReacciones/reaccionesPorCancion

```
package co.edu.unicauca.servidorchat.capaControladores;
```

```
import org.springframework.messaging.handler.annotation.MessageMapping;
```

```
import org.springframework.messaging.handler.annotation.SendTo;
```

```
import org.springframework.stereotype.Controller;
```

```
@Controller
```

```
public class ReaccionesController {
```

```
    @MessageMapping("/enviarReaccion")
```

```
    @SendTo("/brokerDeReacciones/reaccionesPorCancion")
```

```
    public String processMessage(String message) {
```

```
        return message;
```

```
    }
```

```
}
```

Le dice a Spring que esta clase actúa como un controlador especializado para recibir y manejar mensajes STOMP

Esto define el destino de entrada.

message es el contenido que envió el cliente. Puede ser "like", "heart", "fun", etc.

Esto especifica que la respuesta se debe reenviar a todos los suscriptores.

Este archivo configura toda la infraestructura WebSocket/STOMP en el servidor Spring Boot.

```
package co.edu.unicauca.servidorchat.capaConfiguracionWebSocket;

import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import org.springframework.web.socket.config.annotation.*;

→ 1 bean
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/brokerDeReacciones");
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws")
            .setAllowedOriginPatterns("*")
            .withSockJS();
    }
}
```

@EnableWebSocketMessageBroker:
Activa el soporte de WebSocket con protocolo STOMP.

Esto habilita un broker interno de Spring que maneja los mensajes salientes (es decir, hacia los clientes) y soporta suscripciones de los clientes

Cualquier mensaje enviado a destinos que empiecen por: /brokerDeReacciones será manejado por el broker y enviado a TODOS los suscriptores.

```
package co.edu.unicauca.servidorchat.capaConfiguracionWebSocket;
```

```
import org.springframework.context.annotation.Configuration;  
import org.springframework.messaging.simp.config.MessageBrokerRegistry;  
import org.springframework.web.socket.config.annotation.*;
```

→ 1 bean

```
@Configuration
```

```
@EnableWebSocketMessageBroker
```

```
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
```

```
    @Override
```

```
    public void configureMessageBroker(MessageBrokerRegistry config) {
```

```
        config.enableSimpleBroker("/brokerDeReacciones");
```

```
        config.setApplicationDestinationPrefixes("/app");
```

```
    }
```

```
    @Override
```

```
    public void registerStompEndpoints(StompEndpointRegistry registry) {
```

```
        registry.addEndpoint("/ws")
```

```
            .setAllowedOriginPatterns("*")
```

```
            .withSockJS();
```

```
    }
```

```
}
```

Define el prefijo para los mensajes entrantes desde el cliente.

Define el endpoint donde los clientes se conectan:

Permite conexiones desde cualquier dominio (CORS).

Crea compatibilidad con navegadores antiguos

En el archivo application.properties se define el cuerpo de escucha del microservicio

≡ application.properties ×

servidorchat > src > main > resources > ≡ application.properties

```
1  spring.application.name=servidorReacciones
```

```
2  server.port=5000
```

Se deben ejecutar varios clientes.

- a) Cada cliente se conecta con el microservicio de reacciones. Cada cliente envía reacciones al microservicio, el cual las reenvía a los clientes. Observe que si un cliente se desconecta no deben llegar las reacciones.
- b) b) se debe realizar una comparación entre el callback con java RMI y el callback con websockets.

